



PERTEMUAN 8

Classes and Objects

Memahami konsep dasar pemrograman berorientasi objek dalam Java — mulai dari deklarasi kelas, instance variables, konstruktor, hingga studi kasus nyata seperti Account dan GradeBook.



Instance Variables, Set Methods, dan Get Methods

Dalam pemrograman Java berbasis objek, setiap objek memiliki datanya sendiri yang disimpan dalam **instance variables**. Konsep ini menjadi fondasi penting dalam memahami bagaimana sebuah kelas bekerja.

Instance Variables Menyimpan Data Unik

Ketika akun yang berbeda biasanya memiliki nama yang berbeda, kelas `Account` menggunakan variabel `instance name` untuk menyimpan data tersebut. Setiap objek memiliki salinan variabelnya sendiri.

Variabel Instance per Objek

Variabel instance kelas mempertahankan data untuk setiap objek (yaitu, setiap *instance*) dari kelas. Dua objek dari kelas yang sama memiliki memori terpisah untuk instance variables mereka.

Perhatikan Contoh Berikut

Dengan memahami bagaimana `setName()` dan `getName()` bekerja, kita dapat melihat bagaimana data diatur dan diambil dari sebuah objek secara terstruktur dan aman.



Study Case: AccountTest Class

Membuat dan Menggunakan Object dalam Aplikasi Java

Kelas `AccountTest` berfungsi sebagai titik masuk aplikasi Java karena mengandung metode `main`. Kelas `Account` sendiri **tidak dapat dijalankan secara mandiri** karena tidak memiliki metode `main` — jika Anda mengetik `java Account` di command window, Anda akan mendapat pesan error *"Main method not found in class Account"*.

- ① Solusinya: deklarasikan kelas terpisah yang berisi metode `main`, atau tempatkan metode `main` langsung di dalam kelas `Account`.

Pendekatan terbaik adalah memisahkan kelas data (`Account`) dari kelas pengujian (`AccountTest`) agar kode lebih terstruktur dan mudah dipelihara.

Penerapan Class AccountTest

Struktur Kelas AccountTest

Kelas ini berisi metode `main` yang menjadi titik awal eksekusi program. Di sinilah objek dari kelas `Account` dibuat dan semua metodenya dipanggil.

- Membuat objek `Account` menggunakan kata kunci `new`
- Memanggil `setName()` untuk mengatur nama akun
- Memanggil `getName()` untuk menampilkan nama akun

Contoh Alur Eksekusi

01

Buat Objek

```
Account myAccount = new Account();
```

02

Set Nama

```
myAccount.setName("Alice");
```

03

Tampilkan Nama

```
System.out.println(myAccount.getName());
```

Compiling & Eksekusi Multiple Classes

Dalam proyek Java yang melibatkan lebih dari satu kelas, manajemen file menjadi hal yang penting untuk efisiensi kompilasi dan eksekusi program.

Satu Direktori, Lebih Mudah

Untuk mengkompilasi beberapa kelas sekaligus, menempatkan semua file `.java` dalam satu direktori yang sama adalah praktik terbaik. Pemanggilan antar kelas menjadi lebih sederhana.

Folder Berbeda, Tetap Bisa

Meskipun direkomendasikan dalam satu direktori, Java juga mendukung kompilasi kelas dari folder yang berlainan dengan menggunakan `classpath` yang tepat saat kompilasi.

Kompilasi Sekaligus

Gunakan perintah `javac *.java` untuk mengkompilasi semua file Java dalam satu direktori sekaligus, menghemat waktu dan menghindari error dependensi antar kelas.

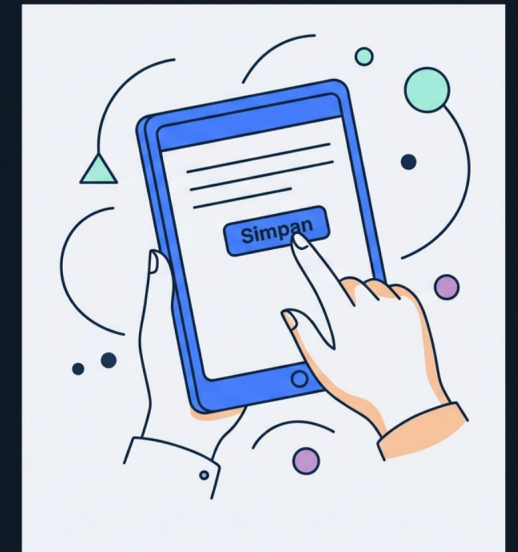
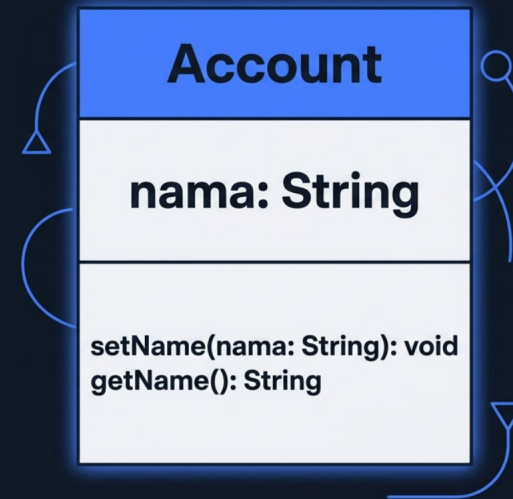


Account UML Class Diagram

Diagram kelas UML adalah alat visual yang membantu pengembang memahami struktur sebuah kelas sebelum kode ditulis. UML (Unified Modeling Language) digunakan secara luas di industri perangkat lunak.

Mengapa UML Penting?

- Memvisualisasikan **atribut** (instance variables) dan **operasi** (metode) kelas secara ringkas
- Membantu perancang sistem mendefinisikan sistem secara **grafis** dan **independen** dari **bahasa pemrograman**
- Menjembatani komunikasi antara analis, desainer, dan programmer
- Memungkinkan perencanaan sistem sebelum implementasi aktual dimulai



Catatan Tambahan: Class AccountTest

"Deklarasi import di baris 3 tidak diperlukan jika kita merujuk ke class Scanner di seluruh file ini sebagai

Software Engineering: Private Instance Variables

Public Set dan Get Methods sebagai Pelindung Data

Dalam desain perangkat lunak yang baik, **enkapsulasi** adalah prinsip utama. Variabel instance `private` disembunyikan di dalam objek dan hanya dapat diakses melalui metode publik yang telah dirancang dengan hati-hati.



Lapisan Dalam (Private)

Variabel instance `name` bersifat `private` — tersembunyi di dalam objek dan tidak dapat diakses langsung dari luar kelas. Ini melindungi integritas data.

Lapisan Luar (Public Methods)

Metode `getName()` dan `setName()` yang bersifat `public` membentuk lapisan pelindung. Kode klien hanya dapat berinteraksi dengan objek melalui metode publik ini.

Manfaat Enkapsulasi

Dengan pola ini, kita dapat memvalidasi data sebelum disimpan, mengontrol cara data dibaca, dan mengubah implementasi internal tanpa memengaruhi kode klien.

Account Class: Initializing Objects with Constructors

Mengapa Konstruktor Diperlukan?

Java membutuhkan **panggilan konstruktor** untuk setiap objek yang dibuat. Ini menjadikan konstruktor sebagai titik ideal untuk menginisialisasi variabel instance objek sejak awal pembuatannya.

Tanpa inisialisasi yang tepat, variabel instance bertipe `String` akan bernilai `null` — yang dapat menyebabkan error di kemudian hari.

Konstruktor dengan Parameter

Contoh berikut meningkatkan kelas `Account` dengan konstruktor yang menerima parameter `name` dan langsung menggunakannya untuk menginisialisasi variabel instance saat objek dibuat.

01

Deklarasi Konstruktor

```
public Account(String name)
```

02

Inisialisasi di Dalam

```
this.name = name;
```

03

Panggil Saat Buat Objek

```
new Account("Jane Green")
```



Declaring an Account Constructor for Custom Initialization

Saat Anda mendeklarasikan kelas, Anda dapat menyediakan **konstruktor kustom** untuk menentukan inisialisasi spesifik bagi objek kelas Anda — jauh lebih fleksibel dibandingkan mengandalkan nilai default.

1 Tentukan Nama Saat Pembuatan

Dengan konstruktor berparameter, Anda bisa langsung menentukan nama objek Account pada saat objek tersebut dibuat, bukan setelahnya melalui metode `setName()`.

2 Konstruktor Menggantikan Default

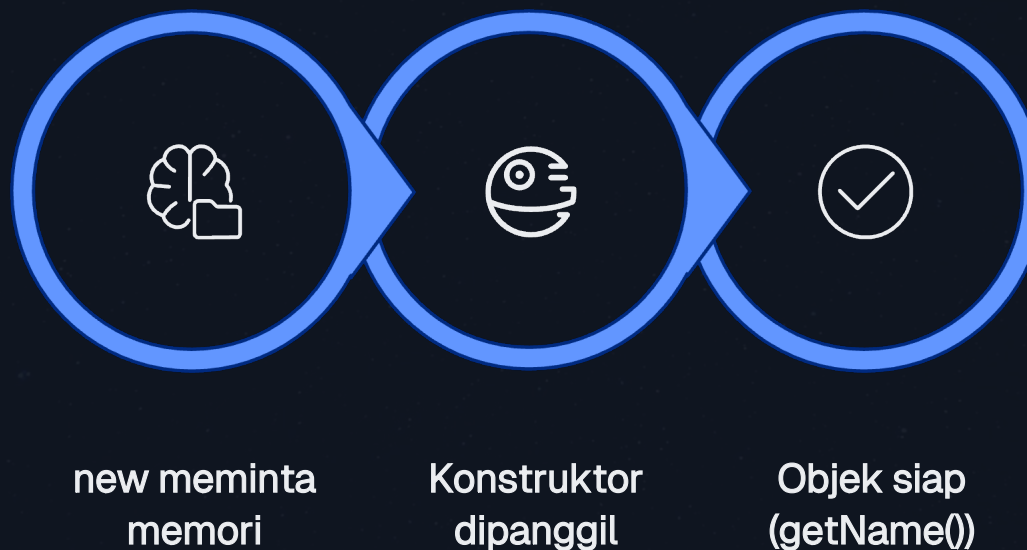
Ketika Anda mendefinisikan konstruktor sendiri, Java **tidak lagi menyediakan** konstruktor default (no-argument). Anda harus mendefinisikannya secara eksplisit jika masih dibutuhkan.

3 Inisialisasi Lebih Aman dan Bersih

Konstruktor kustom memastikan bahwa setiap objek selalu memiliki data yang valid sejak pertama kali dibuat — mengurangi risiko bug akibat objek dengan state yang tidak lengkap.

AccountTest: Initializing Account Objects saat Dibuat

Program `AccountTest` menginisialisasi dua objek `Account` menggunakan konstruktor berparameter.



account1 — Jane Green

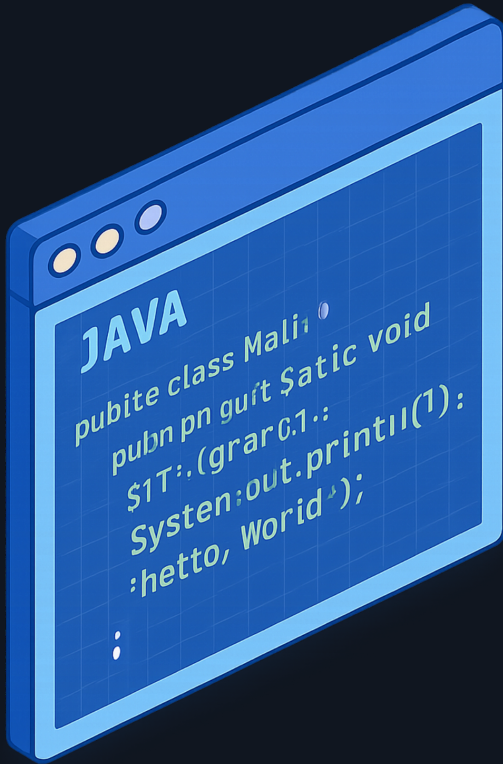
Baris 8: `Account account1 = new Account("Jane Green");` — membuat objek, mengalokasikan memori, dan menginisialisasi nama sekaligus dalam satu pernyataan.

account2 — John Blue

Baris 9: `Account account2 = new Account("John Blue");` — proses yang sama diulang. Setiap objek menyimpan salinan variabel instance `name` miliknya sendiri.

✓ Output program mengonfirmasi bahwa masing-masing objek menyimpan nama yang berbeda, membuktikan independensi data antar instance.

Visualisasi: Dua Objek Account dalam Memori



account1

`name = "Jane Green"`

Objek pertama dengan identitas uniknya sendiri. Perubahan pada `account1` tidak memengaruhi `account2`.

account2

`name = "John Blue"`

Objek kedua dengan data terpisah di memori. Keduanya merupakan instance dari kelas `Account` yang sama.

Referensi Variable

`account1` dan `account2` adalah variabel referensi yang menyimpan alamat objek di memori heap Java, bukan objeknya langsung.

Konsep ini adalah inti dari pemrograman berorientasi objek — setiap instance memiliki **state-nya sendiri** yang independen, meskipun berbagi struktur yang sama dari kelas yang sama.

Account Class dengan Balance

Instance Variable Bertipe `double` untuk Menyimpan Saldo

Aplikasi berikutnya mengembangkan kelas `Account` agar lebih realistis dengan menambahkan variabel instance `balance`. Sebuah bank pada umumnya melayani banyak rekening, masing-masing dengan saldo sendiri.



name: String

Menyimpan nama pemilik akun. Setiap objek `Account` memiliki salinan `name`-nya sendiri yang independen.



balance: double

Menyimpan saldo rekening bertipe `double`. Baris 7 mendeklarasikan variabel instance ini sehingga setiap objek memiliki saldo tersendiri.



Desain Nyata

Pendekatan ini mencerminkan sistem perbankan nyata di mana setiap akun memiliki nama dan saldo yang benar-benar terpisah satu sama lain.



Konstruktor Account dengan Dua Parameter

Konstruktor Baru

Kelas `Account` yang diperbarui memiliki konstruktor dengan dua parameter: `name` (String) dan `initialBalance` (double).

Konstruktor memvalidasi nilai `initialBalance` — jika nilainya lebih besar dari nol, nilai tersebut disimpan; jika tidak, `balance` tetap bernilai `0.0`.

Metode Baru: deposit dan getBalance

→ `deposit(double amount)`

Menambahkan jumlah ke saldo jika `amount` valid (lebih dari nol).

→ `getBalance()`

Mengembalikan nilai `balance` saat ini kepada pemanggil.

→ `getName()`

Mengembalikan nama pemilik akun seperti sebelumnya.

AccountTest Class untuk Menggunakan Class Account

Class `AccountTest` (Gambar 7.9) menciptakan dua objek `Account` dan menguji berbagai skenario termasuk validasi saldo.

Baris 7–8: Buat Dua Objek

Objek pertama dengan saldo valid 50.00, objek kedua dengan saldo tidak valid -7.53 — untuk menguji validasi konstruktor.

1

2

Baris 11–14: Tampilkan Data

Memanggil `getName()` dan `getBalance()` dari masing-masing akun untuk menampilkan nama dan saldo awal.

Deposit dan Verifikasi

Melakukan deposit ke akun dan menampilkan saldo yang diperbarui. Deposit tidak valid (negatif) akan diabaikan oleh metode.

3

4

Output Akhir

Output mengonfirmasi bahwa validasi bekerja — saldo negatif awal diset ke 0.0, dan deposit valid berhasil ditambahkan.

Validasi Data dalam Konstruktordan Metode

Salah satu tanggung jawab utama konstruktor dan metode setter adalah **memvalidasi data** sebelum menyimpannya ke variabel instance.



Validasi Saldo Awal

Jika `initialBalance` lebih besar dari 0.0, nilai tersebut disimpan. Jika tidak, `balance` diinisialisasi ke 0.0 secara otomatis.



Validasi Deposit

Metode `deposit()` memeriksa apakah jumlah yang disetor lebih besar dari nol sebelum menambahkannya ke saldo. Nilai negatif diabaikan.



Data Selalu Konsisten

Dengan validasi ini, objek `Account` selalu berada dalam **state yang valid** — saldo tidak pernah bernilai negatif berkat penjagaan di konstruktor dan metode.

UML Class Diagram for Class Account (with Balance)

Diagram UML berikut merepresentasikan struktur lengkap kelas Account yang telah diperkaya dengan variabel instance balance dan metode-metode barunya.



Notasi UML

- Tanda - berarti akses private
- Tanda + berarti akses public

Manfaat Diagram

Diagram ini memungkinkan tim pengembang memahami desain kelas sebelum menulis satu baris kode pun, mendorong kolaborasi yang lebih efektif.

Case Study: Card Shuffling dan Dealing Simulation

Kelas Card Merepresentasikan Sebuah Kartu Remi

Studi kasus ini mensimulasikan sistem kartu remi digital menggunakan konsep kelas dan objek Java. Setiap kartu direpresentasikan oleh sebuah objek dengan atribut `face` (nilai kartu) dan `suit` (jenis kartu).

Kelas Card

Merepresentasikan satu kartu remi dengan dua atribut: `face` (As, 2–10, J, Q, K) dan `suit` (Sekop, Hati, Wajik, Keriting). Konstruktornya menerima kedua nilai tersebut.

Kelas DeckOfCards

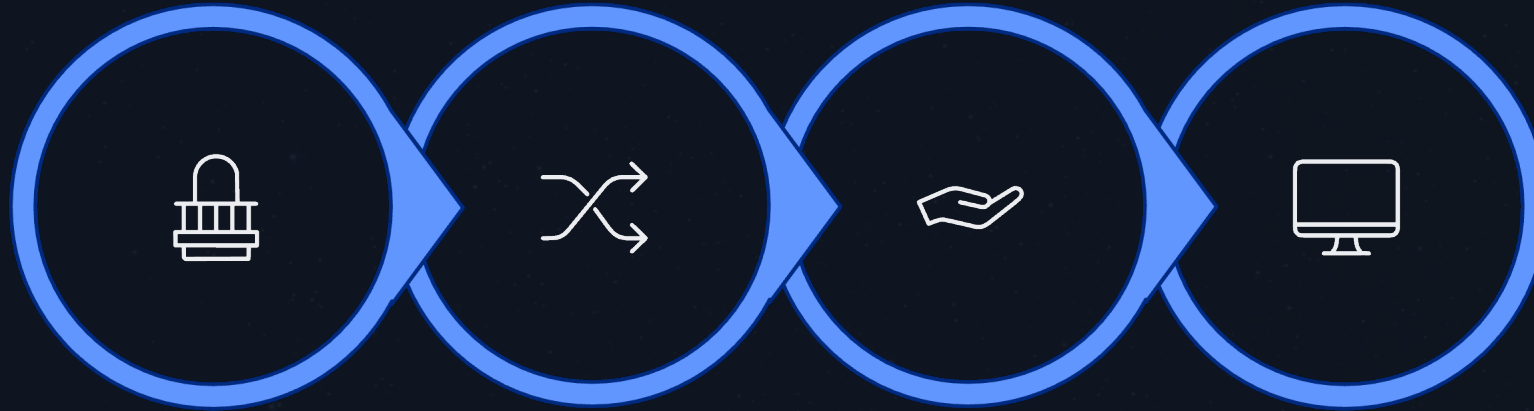
Merepresentasikan satu set lengkap 52 kartu. Kelas ini berisi array objek `Card` dan metode untuk mengacak (`shuffle()`) serta membagikan (`dealCard()`) kartu.

Prinsip OOP yang Diterapkan

Studi kasus ini menggabungkan array objek, enkapsulasi, dan interaksi antar objek — contoh nyata dari kekuatan pemrograman berorientasi objek dalam Java.



Card Shuffling dan Dealing



Inisialisasi

Shuffle

Deal

Tampilkan

Algoritma Shuffle

Menggunakan metode `Collections.shuffle()` atau algoritma Fisher-Yates untuk memastikan setiap permutasi 52 kartu memiliki probabilitas kemunculan yang sama — menghasilkan pengacakan yang benar-benar acak.

Proses Dealing

Setiap panggilan ke `dealCard()` mengembalikan objek `Card` berikutnya dari tumpukan. Setelah 52 kartu dibagikan, tidak ada kartu lagi yang tersisa — mencerminkan perilaku dek kartu nyata.

Case Study: GradeBook

Menggunakan Array untuk Menyimpan Nilai

Studi kasus ini mengembangkan kelas `GradeBook` yang digunakan instruktur untuk mengelola nilai siswa secara komprehensif dan menghasilkan laporan statistik yang informatif.



Penyimpanan Nilai

Versi pertama menggunakan **array satu dimensi** untuk menyimpan nilai ujian tunggal. Versi lanjutannya menggunakan **array dua dimensi** untuk nilai dari beberapa ujian.



Laporan Nilai

Program menghasilkan laporan lengkap yang mencakup: semua nilai individual, **rata-rata kelas**, nilai tertinggi, nilai terendah, dan **diagram batang distribusi nilai**.



Aplikasi Nyata

Studi kasus ini mensimulasikan sistem penilaian nyata di lingkungan akademik, menunjukkan bagaimana Java dapat digunakan untuk membangun aplikasi manajemen data yang fungsional.



Terima Kasih!

Semoga materi **Classes and Objects** pada Pertemuan 8 ini memberikan pemahaman yang kuat tentang fondasi pemrograman berorientasi objek dalam Java.



Poin Utama

- Instance variables menyimpan data per objek
- Konstruktor menginisialisasi objek saat dibuat
- `private` + getter/setter = enkapsulasi

Langkah Selanjutnya

- Coba implementasikan kelas `Account` sendiri
- Eksplorasi studi kasus `GradeBook`
- Pelajari konsep **inheritance** dan **polymorphism**

Sasa Ani Arnomo, S.Kom., M.S.I. | Universitas Putera Batam | Pemrograman Java